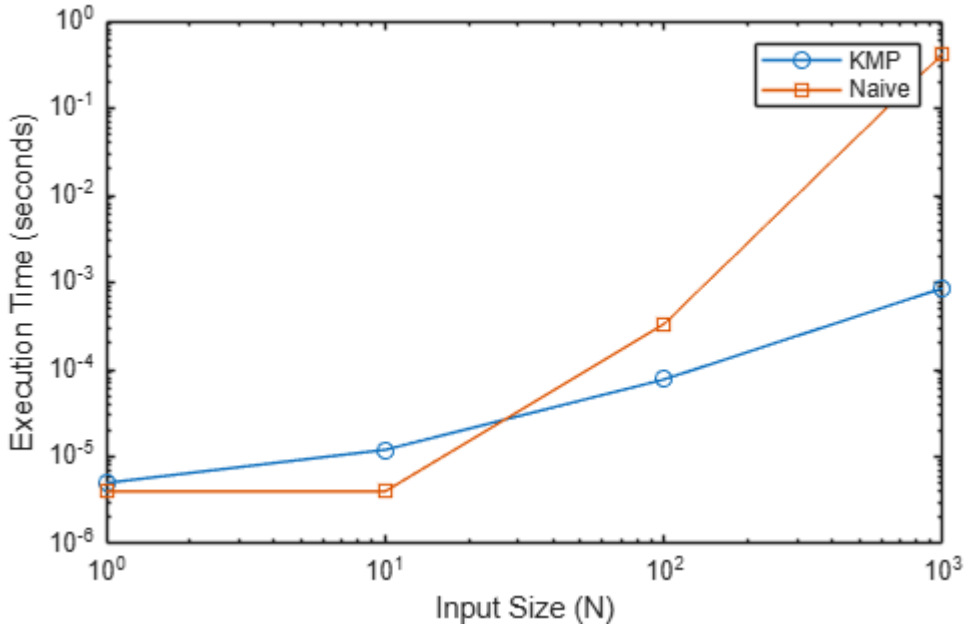
Spencer Ng. Project 1.

Orientation:

For this project, I decided to take on LeetCode problem 686, which has been a problem that I have been unable to solve. Upon looking at some guides, I learned that an in-depth solution would require using the KMP algorithm, which I didn't understand at the time. Therefore, the goal of this project is to learn the KMP algorithm and to apply it to LeetCode problem 686. To ensure that I have mastered the KMP algorithm, I will also solve an additional LeetCode KMP problem, provide a run time graph of my old problem 686 solution, which was denied to exceeding time limits, to my new solution, and to provide an explanation as to why the new solution has a superior run time.

```
KMP = [0.000005, 0.000012, 0.000077, 0.000862];
naive = [0.000004, 0.000004, 0.000333, 0.424351]; size = [1, 10, 100, 1000];
figure(); loglog(size, KMP, '-o', size, naive, '-s')
xlabel("Input Size (N)"); ylabel("Execution Time (seconds)"); legend("KMP", "Naive")
```



A quick note for input sizes: in order to simplify the code and to graph this on a 2D graph, the input size assumes that $N = M$, where M is the size of string a, and N the size of string b. In other words, both input strings have the same size.

As seen from the run time graph, the KMP-based solution is significantly faster for larger input sizes, with negligible run time differences for smaller input sizes. The KMP-based solution has a run time of $O(N + M)$, while the naive algorithm has a run time of $O(N^2M)$. The naive solution first goes through a while loop that can at most execute N times. The other condition of the while loop is *b not in a * i*. String multiplication has a run time of the size of the string multiplied by the number of times the string is multiplied, so $O(iM)$ is the run

time since the number of times the string is multiplied depends on the current value of the iteration. The *not in* portion of the code has a run time equivalent to the sum of the lengths of the two strings, so the run time of this portion will be $O(iM + N)$. When combining the fact that this will execute a maximum of N times, the run time comes out to $O(N^2M)$ as a result of the arithmetic progression sum $(M + 2M + ... + NM)$. This is much slower than the KMP-based solution. The KMP-based solution starts by creating the longest proper prefix (LPS) table, which is the same length of b, giving this portion a run time of $O(N)$. After creating the table, the code starts the string matching process which, unlike the naive algorithm, doesn't require the iterator to completely reset thanks to the LPS table, which removes redundant operations, giving this portion a run time of $O(N + M)$, as the length is bounded by *aLong* of maximum length $N + M$. The KMP algorithm is only called twice, so the run time for the KMP-based solution is $O(N + M)$.

Solutions for my LeetCode problems can be found on my GitHub.